

Welcome to the KodeKloud Hands-On lab

_ _ _ _ _
 / / _ / _ \ _ \ _ / / _ / / / _ \ _ \
 / , < / / / / / _ / / , < / / / / / / / / /
 / / / / _ / / _ / / _ / / | / / _ / / _ / / _ / /
 / _ / | \ _ _ / _ _ / _ _ / / _ / | / _ _ ^ _ ^ _ / _ _ /

All rights reserved

$$z \rightarrow ls$$

Dockerfile-mysql Dockerfile-ubuntu app

Dockerfile-python Dockerfile-wordpress

```
~ → cat Dockerfile-mysql
```

FROM debian:stretch-slim

```
# add our user and group first to make sure their IDs get assigned consistently, regardless
of whatever dependencies get added
```

```
RUN groupadd -r mysql && useradd -r -g mysql mysql
```

```
RUN apt-get update && apt-get install -y --no-install-recommends gnupg dirmngr && rm -rf /var/lib/apt/lists/*
```

```
# add gosu for easy step-down from root
```

ENV GOSU_VERSION 1.7

```
RUN set -x \
```

```
&& apt-get update && apt-get install -y --no-install-recommends ca-certificates wget  
&& rm -rf /var/lib/apt/lists/* \
```

```
&& wget -O /usr/local/bin/gosu  
"https://github.com/tianon/gosu/releases/download/$GOSU_VERSION/gosu-$(dpkg --  
print-architecture)" \
```

```
&& wget -O /usr/local/bin/gosu.asc  
"https://github.com/tianon/gosu/releases/download/$GOSU_VERSION/gosu-$(dpkg --  
print-architecture).asc" \
```

```
&& export GNUPGHOME="$(mktemp -d)" \
```

```
&& gpg --batch --keyserver ha.pool.sks-keyservers.net --recv-keys  
B42F6819007F00F88E364FD4036A9C25BF357DD4 \
```

```
&& gpg --batch --verify /usr/local/bin/gosu.asc /usr/local/bin/gosu \
```

```
&& gpgconf --kill all \
```

```
&& rm -rf "$GNUPGHOME" /usr/local/bin/gosu.asc \
```

```
&& chmod +x /usr/local/bin/gosu \
```

```
&& gosu nobody true \
```

```
&& apt-get purge -y --auto-remove ca-certificates wget
```

```
RUN mkdir /docker-entrypoint-initdb.d
```

```
RUN apt-get update && apt-get install -y --no-install-recommends \
```

```
# for MYSQL_RANDOM_ROOT_PASSWORD
```

```
pwgen \
```

```
# for mysql_ssl_rsa_setup
```

```
openssl \
```

```
# FATAL ERROR: please install the following Perl modules before executing  
/usr/local/mysql/scripts/mysql_install_db:
```

```
# File::Basename
```

```
# File::Copy
```

```
# Sys::Hostname
```

```
# Data::Dumper
```

```
perl \
```

```
&& rm -rf /var/lib/apt/lists/*
```

```
RUN set -ex; \
```

```
# gpg: key 5072E1F5: public key "MySQL Release Engineering <mysql-build@oss.oracle.com>" imported
```

```
key='A4A9406876FCBD3C456770C88C718D3B5072E1F5'; \
```

```
export GNUPGHOME="$(mktemp -d)"; \
```

```
gpg --batch --keyserver ha.pool.sks-keyservers.net --recv-keys "$key"; \
```

```
gpg --batch --export "$key" > /etc/apt/trusted.gpg.d/mysql.gpg; \
```

```
gpgconf --kill all; \
```

```
rm -rf "$GNUPGHOME"; \
```

```
apt-key list > /dev/null
```

```
ENV MYSQL_MAJOR 8.0
```

```
ENV MYSQL_VERSION 8.0.17-1debian9
```

```
RUN echo "deb http://repo.mysql.com/apt/debian/ stretch mysql-${MYSQL_MAJOR}" >  
/etc/apt/sources.list.d/mysql.list
```

```
# the "/var/lib/mysql" stuff here is because the mysql-server postinst doesn't have an  
explicit way to disable the mysql_install_db codepath besides having a database already  
"configured" (ie, stuff in /var/lib/mysql/mysql)
```

```
# also, we set debconf keys to make APT a little quieter
```

```

RUN {\
    echo mysql-community-server mysql-community-server/data-dir select "; \
    echo mysql-community-server mysql-community-server/root-pass password "; \
    echo mysql-community-server mysql-community-server/re-root-pass password "; \
    echo mysql-community-server mysql-community-server/remove-test-db select
false; \

    } | debconf-set-selections \

    && apt-get update && apt-get install -y mysql-community-client="${MYSQL_VERSION}"
mysql-community-server-core="${MYSQL_VERSION}" && rm -rf /var/lib/apt/lists/* \

    && rm -rf /var/lib/mysql && mkdir -p /var/lib/mysql /var/run/mysqld \

    && chown -R mysql:mysql /var/lib/mysql /var/run/mysqld \

# ensure that /var/run/mysqld (used for socket and lock files) is writable regardless of the
UID our mysqld instance ends up having at runtime

    && chmod 777 /var/run/mysqld

```

```
VOLUME /var/lib/mysql
```

```
# Config files
```

```
COPY config/ /etc/mysql/
```

```
COPY docker-entrypoint.sh /usr/local/bin/
```

```
RUN ln -s usr/local/bin/docker-entrypoint.sh /entrypoint.sh # backwards compat
```

```
ENTRYPOINT ["docker-entrypoint.sh"]
```

```
EXPOSE 3306 33060
```

```
CMD ["mysqld"]
```

```
~ → ls
```

```
Dockerfile-mysql  Dockerfile-ubuntu  app
```

Dockerfile-python Dockerfile-wordpress

~ → cat Dockerfile-wordpress

FROM php:7.1-apache

install the PHP extensions we need

(<https://make.wordpress.org/hosting/handbook/handbook/server-environment/#php-extensions>)

RUN set -ex; \

\

savedAptMark="\$(apt-mark showmanual)"; \

\

apt-get update; \

apt-get install -y --no-install-recommends \

libjpeg-dev \

libmagickwand-dev \

libpng-dev \

; \

\

docker-php-ext-configure gd --with-png-dir=/usr --with-jpeg-dir=/usr; \

docker-php-ext-install -j "\$(nproc)" \

bcmath \

exif \

gd \

mysqli \

opcache \

```

zip \
; \
pecl install imagick-3.4.4; \
docker-php-ext-enable imagick; \
\
# reset apt-mark's "manual" list so that "purge --auto-remove" will remove all build
dependencies

apt-mark auto '*.*' > /dev/null; \
apt-mark manual $savedAptMark; \
ldd "$(php -r 'echo ini_get("extension_dir");')/*.*.so \
| awk '/=>/ { print $3 }' \
| sort -u \
| xargs -r dpkg-query -S \
| cut -d: -f1 \
| sort -u \
| xargs -rt apt-mark manual; \
\
apt-get purge -y --auto-remove -o APT::AutoRemove::RecommendsImportant=false; \
rm -rf /var/lib/apt/lists/*

# set recommended PHP.ini settings
# see https://secure.php.net/manual/en/opcache.installation.php
RUN { \
    echo 'opcache.memory_consumption=128'; \
    echo 'opcache.interned_strings_buffer=8'; \
    echo 'opcache.max_accelerated_files=4000'; \

```

```
    echo 'opcache.revalidate_freq=2'; \

    echo 'opcache.fast_shutdown=1'; \

} > /usr/local/etc/php/conf.d/opcache-recommended.ini

# https://wordpress.org/support/article/editing-wp-config-php/#configure-error-logging

RUN { \

# https://www.php.net/manual/en/errorfunc.constants.php

# https://github.com/docker-library/wordpress/issues/420#issuecomment-517839670

    echo 'error_reporting = E_ERROR | E_WARNING | E_PARSE | E_CORE_ERROR | \
E_CORE_WARNING | E_COMPILE_ERROR | E_COMPILE_WARNING | \
E_RECOVERABLE_ERROR'; \

    echo 'display_errors = Off'; \

    echo 'display_startup_errors = Off'; \

    echo 'log_errors = On'; \

    echo 'error_log = /dev/stderr'; \

    echo 'log_errors_max_len = 1024'; \

    echo 'ignore_repeated_errors = On'; \

    echo 'ignore_repeated_source = Off'; \

    echo 'html_errors = Off'; \

} > /usr/local/etc/php/conf.d/error-logging.ini
```

RUN a2enmod rewrite expires

VOLUME /var/www/html

ENV WORDPRESS_VERSION 5.2.2

ENV WORDPRESS_SHA1 3605bcbe9ea48d714efa59b0eb2d251657e7d5b0

```
RUN set -ex; \
```

```
    curl -o wordpress.tar.gz -fSL "https://wordpress.org/wordpress-  
${WORDPRESS_VERSION}.tar.gz"; \
```

```
    echo "$WORDPRESS_SHA1 *wordpress.tar.gz" | sha1sum -c -; \
```

```
# upstream tarballs include ./wordpress/ so this gives us /usr/src/wordpress
```

```
    tar -xzf wordpress.tar.gz -C /usr/src/; \
```

```
    rm wordpress.tar.gz; \
```

```
    chown -R www-data:www-data /usr/src/wordpress
```

```
COPY docker-entrpoint.sh /usr/local/bin/
```

```
ENTRYPOINT ["docker-entrpoint.sh"]
```

```
CMD ["apache2-foreground"]
```

```
~ → ls
```

```
Dockerfile-mysql  Dockerfile-ubuntu  app
```

```
Dockerfile-python  Dockerfile-wordpress
```

```
~ → cat Dockerfile-ubuntu
```

```
#
```

```
# Ubuntu Dockerfile
```

```
#
```

```
# https://github.com/dockerfile/ubuntu
```

```
#
```

```
# Pull base image.
```


FROM ubuntu:14.04

Install.

RUN \

```
sed -i 's/# \(*multiverse$\)/\1/g' /etc/apt/sources.list && \
apt-get update && \
apt-get -y upgrade && \
apt-get install -y build-essential && \
apt-get install -y software-properties-common && \
apt-get install -y byobu curl git htop man unzip vim wget && \
rm -rf /var/lib/apt/lists/*
```

Add files.

ADD root/.bashrc /root/.bashrc

ADD root/.gitconfig /root/.gitconfig

ADD root/.scripts /root/.scripts

Set environment variables.

ENV HOME /root

Define working directory.

WORKDIR /root

Define default command.

CMD ["bash"]

~ → docker run -d ubuntu sleep 1000

db3679183d1c5d3c0290d8cfb8cdbe4a368901d4953f79499a754586292107bf